```
    self.content_type = picture_field.content_type.chomp
    self.data         = picture_field.read
  end

  def base_part_of(file_name)
    File.basename(file_name).gsub(/[^\w._-]/, '')
  end
end
```

We define an accessor called `uploaded_picture=` to receive the file uploaded by the form. The object returned by the form is an interesting hybrid. It's file-like, so we can read its contents with the `read` method; that's how we get the image data into the `data` column. It also has the attributes `content_type` and `original_filename`, which let us get at the uploaded file's metadata. Accessor methods pick all this apart, resulting in a single object stored as separate attributes in the database.

Note that we also add a validation to check that the content type is of the form `image/`*xxx*. We don't want someone uploading JavaScript.

The `save` action in the controller is totally conventional:

rails7/e1/views/app/controllers/upload_controller.rb

```
def save
  @picture = Picture.new(picture_params)
  if @picture.save
    redirect_to(action: 'show', id: @picture.id)
  else
    render(action: :get)
  end
end
```

Now that we have an image in the database, how do we display it? One way is to give it its own URL and link to that URL from an image tag. For example, we could use a URL such as `upload/picture/123` to return the image for picture 123. This would use `send_data` to return the image to the browser. Note how we set the content type and filename—this lets browsers interpret the data and supplies a default name should the user choose to save the image:

rails7/e1/views/app/controllers/upload_controller.rb